TEXAS A&M UNIVERSITY
ROBOTICS TEAM & LEADERSHIP EXPERIENCE

Week 9: ROS 2 Integration and Final System Demo

TURTLE Incubator - Gazebo Simulation Lab

Agenda

ROS 2 Architecture

- Nodes, topics, services, actions, and the DDS middleware

ROS 2 Tools

- RViz, rqt, rosbag, ros2 topic/node/launch commands

Docker and Containers

- How ROS2 and Gazebo run in Docker

Gazebo Built-In Worlds

- Four simulation environments you launch today

Hands-On Exploration

- Launch each world, explore joints and teleop controls

Deliverable

- Four worlds launched, ROS 2 topics inspected, screenshots taken

Deliverable

- Complete all four Gazebo world checkpoints

What You Are Building Today

Today you explore four Gazebo built-in simulation worlds:

- 1 Launch each of the four built-in Gazebo worlds using gz sim
- 2 Control a Panda arm and NAO humanoid with the joint position GUI
- 3 Fly a quadcopter with keyboard teleop commands
- 4 Drive the Tugbot robot through a warehouse environment
- 5 Inspect ROS 2 topics bridged from Gazebo using ros2 topic list
- 6 Echo joint state messages and teleop command topics in real time
- 7 Take a screenshot of each world and annotate what you observe

You only need Docker + VcXsrv running on your laptop — no physical arm required.

ROS 2 Architecture

What Is ROS 2?

ROS 2 (Robot Operating System 2) is a middleware framework for building robot software systems.

What it gives you:

- A standard way for software modules (nodes) to communicate over a network
- A rich ecosystem of libraries: navigation, perception, manipulation, simulation
- Tools for visualization, debugging, recording, and replaying robot data

Why we use it:

- Every professional robotics team uses ROS 2 or something built on it
- It handles inter-process communication so you can focus on algorithms
- Your Week 4-8 scripts become nodes that talk to each other automatically

ROS 2 vs ROS 1:

- ROS 2 supports real-time, multi-robot, and security - ROS 1 is legacy and end-of-life in 2025

Nodes, Topics, Services, and Actions

Four communication primitives - each for a different type of interaction:

Nodes

- Independent processes that run your code. One node per logical function.
- `hardware_node`, `filter_node`, `kinematics_node`, `controller_node`, `planner_node`

Topics

- One-to-many publish/subscribe streams. Publisher does not know who is listening.
- Use for: continuous data - joint states, sensor readings, commanded positions

Services

- Request/response. Client sends a request, server replies once.
- Use for: one-off queries - get current pose, reset encoder, check status

Actions

- Long-running tasks with feedback and cancellation support
- Use for: move to goal, execute trajectory, pick-and-place sequence

SO-ARM ROS 2 Node Graph

Five nodes, all connected by topics:

hardware_node
- Reads encoders, sends motor commands

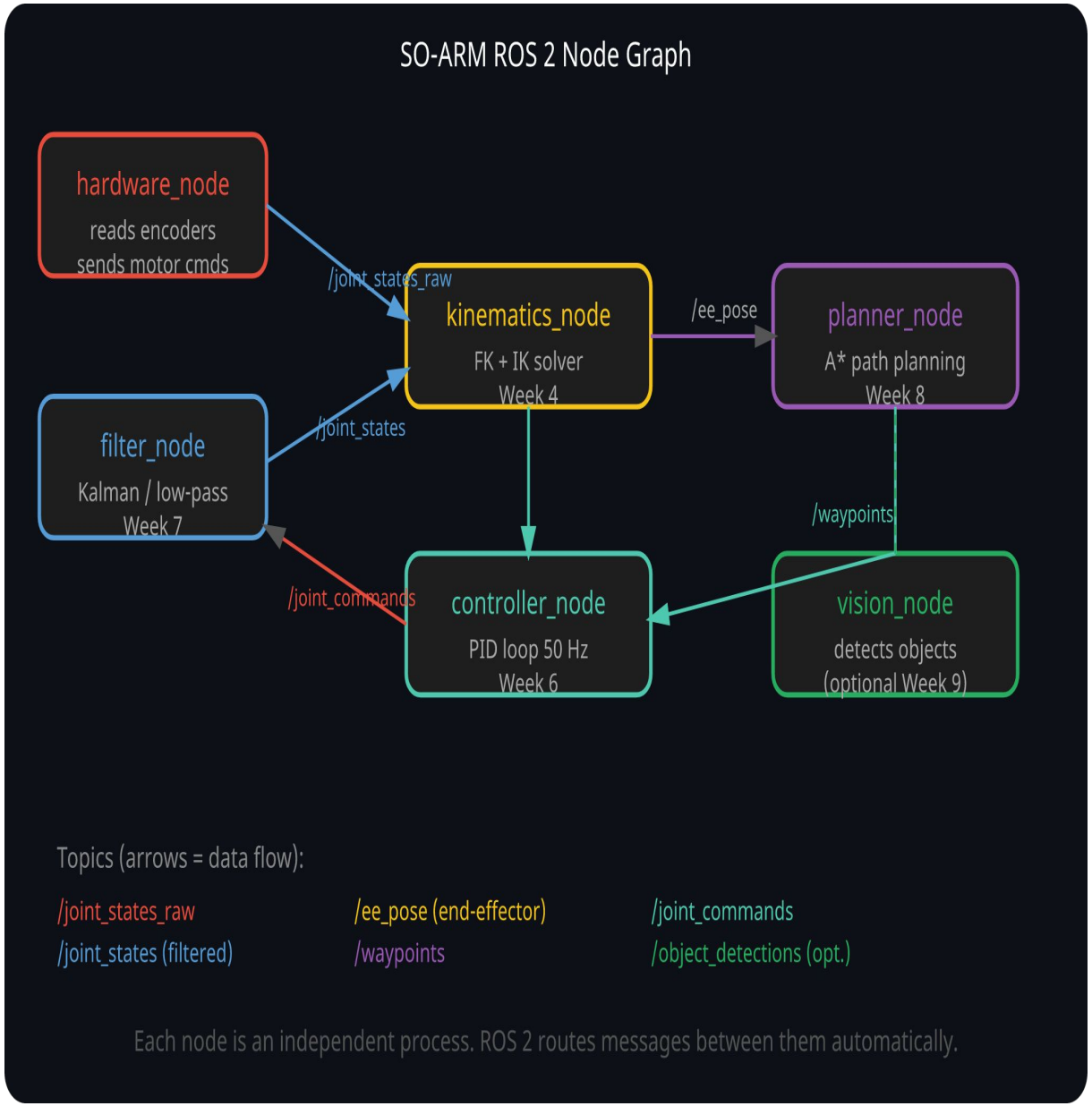
filter_node
- Kalman/low-pass on joint_states_raw

kinematics_node
- FK + IK, publishes ee_pose

controller_node
- PID at 50 Hz, tracks waypoints

planner_node
- A* path planning, publishes waypoints

Each node is a Python file. ROS 2 routes the messages.



DDS - How ROS 2 Actually Moves Data

Under the hood ROS 2 uses DDS (Data Distribution Service) as its transport layer:

What DDS does:

- Discovers other nodes automatically - no central broker needed
- Routes messages between publishers and subscribers by topic name
- Supports QoS (Quality of Service) settings: reliability, latency, history

QoS presets you will use:

- RELIABLE - guaranteed delivery, used for commands and critical data
- BEST_EFFORT - low latency, used for sensor streams where dropping a packet is fine

For the SO-ARM:

- All nodes run on the same machine inside Docker - DDS uses shared memory (fast)
- Use RELIABLE QoS for /joint_commands, BEST_EFFORT for /joint_states

You don't configure DDS directly - ROS 2 handles it. Just match QoS between pub and sub.

ROS 2 Tools

Essential ROS 2 CLI Commands

These commands are your debugger. Learn them before touching any code.

ros2 node list

- List all running nodes - verify your launch brought everything up

ros2 topic list / ros2 topic echo /joint_states

- List all topics. Echo prints every message on a topic in real time.

ros2 topic hz /joint_states

- Measure publish rate - verify your 50 Hz control loop is actually running at 50 Hz

ros2 topic pub /joint_commands std_msgs/Float64MultiArray '{data: [45,0,0,0,0]}'

- Manually publish a message to a topic - great for testing individual nodes

ros2 service call /get_pose ...

- Call a service from the command line - useful for one-shot queries

RViz and rqt

RViz and rqt are your two main GUI debugging tools:

RViz (ROS Visualization)

- 3D visualizer that subscribes to ROS topics and renders them
- Display the SO-ARM URDF model with live joint angles from /joint_states
- Show the end-effector pose, planned path, and obstacle markers in 3D
- `Run: rviz2 -d so_arm.rviz`

rqt

- Plugin-based GUI for inspecting and interacting with ROS data
- rqt_graph: visualize the node graph and all active topic connections
- rqt_plot: live-plot any numeric topic - great for PID tuning
- rqt_console: filter and search through all node log messages
- `Run: rqt  then Plugins menu to add panels`

rosbag - Record and Replay

rosbag records all topic messages to a file you can replay later:

Recording

- `ros2 bag record /joint_states /joint_commands /ee_pose /waypoints`
- Creates a .db3 file with timestamped messages from all specified topics

Replaying

- `ros2 bag play so_arm_run.db3`
- Re-publishes all recorded messages at the original timing
- Lets you replay a pick-and-place run and analyze it without the physical arm

Inspecting

- `ros2 bag info so_arm_run.db3  - shows duration, topics, message counts`

Why this matters: rosbag is how you share runs with teammates, debug failures after the fact, and prove your system worked.

Docker and Containers

Docker and Containers for ROS2

Docker isolates ROS2 and Gazebo from your laptop's OS — not the SO-ARM:

Why Docker for ROS2 / Gazebo:

- ROS2 Humble + Gazebo Classic have complex system dependencies that break on most laptops
- Simulation needs no USB passthrough — so the Docker issues we hit with the arm don't apply here

The SO-ARM runs natively (conda) — Docker is for simulation only:

- SO-ARM uses LeRobot + conda on your laptop — no container needed
- USB/serial passthrough in Docker is unreliable across Mac/Win/Linux
- Docker is only for this week's Gazebo simulation environment

Key Docker commands for this week:

- `docker run -it osrf/ros:humble-desktop bash` - start ROS2 container
- `docker exec -it si_ros2 bash` - shell into running container
- `docker start si_ros2` - restart after a crash
- Files in /root/ros2_ws inside the container persist via a mounted volume.

Gazebo Simulation

Four Built-In Gazebo Worlds

All four worlds ship with Gazebo Harmonic — no download needed:

[1] Panda Joint Control World

- Control a 7-DOF Franka Panda arm with a joint position slider GUI

[2] NAO Joint Control World

- Control a full humanoid (NAO) robot with joint sliders

[3] Quadcopter Teleop World

- Fly a drone with keyboard teleop — W/S forward/back, A/D strafe

[4] Tugbot In Warehouse World

- Drive a differential-drive robot through a warehouse

Launch command for all worlds:

- gz sim then select world from the Quick Start dialog

Required on Windows (before launching):

World 1: Panda Joint Control

What it is

- 7-DOF Franka Panda arm — the industry standard for research manipulation
- Joint position control GUI: drag sliders to set each joint angle
- Physics-accurate simulation: inertia, gravity, joint limits all modeled

How to launch

```
gz sim
# Select 'Panda Joint Control World' from Quick Start
# Press ▶ Play, then drag the sliders on the right
```

ROS 2 topics

```
ros2 topic list          # see all Panda topics
ros2 topic echo /panda/joint_states
```

What to look for

- Joint names: panda_joint1 through panda_joint7
- Position values change as you move sliders
- The arm respects gravity — release a slider and it holds position

World 2: NAO Joint Control

What it is

- NAO: a 58-DOF humanoid robot by SoftBank Robotics
- Used in research, education, and RoboCup soccer competitions
- Same joint-slider GUI as Panda — but way more joints to explore

How to launch

```
gz sim
# Select 'NAO Joint Control' from Quick Start
# Press ▶ Play, then drag head / arm / leg sliders
```

ROS 2 topics

```
ros2 topic list          # nao topics show up here
ros2 topic echo /nao/joint_states
```

What to look for

- Many more joints than Panda — head, shoulders, elbows, wrists, hips, knees, ankles
- Try moving the head pan/tilt — watch it track in 3D
- Compare joint naming convention to Panda — different robot, same ROS 2 interface

World 3: Quadcopter Teleop

What it is

- X3 quadcopter with keyboard teleop and ROS 2 velocity commands
- Demonstrates /cmd_vel Twist messages — the universal robot motion interface
- Hover, translate, and yaw using keyboard controls

How to launch

```
gz sim
# Select 'Quadcopter Teleop' from Quick Start
# Press ▶ Play
```

Keyboard controls

```
W / S    forward / backward
A / D    strafe left / right
Q / E    yaw left / right
R / F    ascend / descend
```

Inspect the velocity topic

```
ros2 topic echo /X3/cmd_vel
# Move the drone -- watch linear.x, linear.y, angular.z
```

World 4: Tugbot In Warehouse

What it is

- Tugbot: a differential-drive ground robot in a full warehouse environment
- Navigates between shelves — realistic autonomous robot scenario
- Pre-programmed navigation path; also supports manual keyboard control

How to launch

```
gz sim
# Select 'Tugbot In Warehouse' from Quick Start
# Press ▶ Play -- robot starts moving on its own
```

ROS 2 topics to explore

```
ros2 topic list
ros2 topic echo /cmd_vel          # drive commands
ros2 topic echo /odom             # odometry
ros2 topic echo /scan             # LIDAR scans
```

What to look for

- /scan gives LIDAR data — same format as a real robot sensor
- /odom publishes position and velocity estimate from wheel encoders
- Compare with NAO/Panda — ground robot has very different topic set

Final Demonstration

Live Demo - Four Gazebo Worlds

Watch each world launch and see how Gazebo connects to ROS 2.

What we will show:

- 1. export DISPLAY=host.docker.internal:0 && export LIBGL_ALWAYS_SOFTWARE=1
- 2. gz sim -- opens Quick Start dialog with all four worlds listed
- 3. Panda Joint Control -- drag sliders, watch arm move in 3D
- 4. ros2 topic list — see /panda/joint_states and other bridged topics
- 5. NAO Joint Control -- control humanoid joints with the same GUI
- 6. Quadcopter Teleop -- fly with keyboard, observe /cmd_vel topic
- 7. Tugbot In Warehouse -- navigate through shelves, inspect /odom
- 8. ros2 topic echo on any topic to see live data flowing from Gazebo

Hands-On

Hands-On - Step 1: Launch Panda World and Explore Panda

Goal: get Gazebo open with the Panda arm world running in Docker

1. **Open Docker and start the container**

```
docker start ros2_jazzy && docker exec -it ros2_jazzy bash
```

2. **Set the display environment variables**

```
export DISPLAY=host.docker.internal:0
export LIBGL_ALWAYS_SOFTWARE=1
```

3. **Launch Gazebo (the Quick Start dialog opens)**

```
gz sim
```

Select 'Panda Joint Control World' and press Play (triangle button)

4. **Verify the arm is visible and physics is running**
   - The yellow Panda arm should appear in the 3D viewport
   - Drag a joint slider on the right panel -- the arm should move

5. **Inspect the joint state topic**

```
ros2 topic echo /panda/joint_states
```

   - You should see joint names and position values updating

Hands-On - Step 2: NAO, Quadcopter, and Tugbot

Goal: explore the remaining three Gazebo worlds

1. **NAO Joint Control World**

   ```
   gz sim   # select 'NAO Joint Control' in Quick Start
   ```

   - Try moving the head pan/tilt and arm joints

   ```
   ros2 topic echo /nao/joint_states
   ```

2. **Quadcopter Teleop World**

   ```
   gz sim   # select 'Quadcopter Teleop'
   ```

   - W/S = forward/back, A/D = strafe, R/F = up/down

   ```
   ros2 topic echo /X3/cmd_vel
   ```

3. **Tugbot In Warehouse World**

   ```
   gz sim   # select 'Tugbot In Warehouse'
   ```

   - Press Play and watch it navigate autonomously

   ```
   ros2 topic echo /scan
   ros2 topic echo /odom
   ```

Common Mistakes

3 Errors You Will Definitely Hit

Every student hits these in Docker on Windows:

Error 1 - Gazebo window never appears

- VcXsrv is not running, or 'Disable access control' was not checked

Fix: restart VcXsrv with 'Disable access control' checked

```
export DISPLAY=host.docker.internal:0
export LIBGL_ALWAYS_SOFTWARE=1
```

Error 2 - 'gz: command not found'

- Gazebo environment is not sourced in the container

Fix: source ROS 2 setup first

```
source /opt/ros/jazzy/setup.bash
```

Error 3 - Blank gray viewport / no physics

- Forgot to press Play after selecting a world

Fix: click the triangle Play button in the toolbar

Deliverable

Deliverable - Four Gazebo Worlds Explored

Show all four checkpoints to an instructor during lab:

[+] Panda Joint Control running -- arm moves when you drag sliders

Run: ros2 topic echo /panda/joint_states (must show live data)

[+] NAO Joint Control running -- at least one limb moved in simulation

Run: ros2 topic echo /nao/joint_states (must show live data)

[+] Quadcopter Teleop -- drone hovering and responding to keyboard

Run: ros2 topic echo /X3/cmd_vel (must update on keypresses)

[+] Tugbot In Warehouse -- robot navigating autonomously on Play

Run: ros2 topic echo /scan (must show LIDAR data)

You Built a Robotics Software Stack

What You Accomplished This Semester

Nine weeks ago you cloned a repo and ran your first Python script.

Week 1 - Conda environment, Python basics, GitHub
Week 2 - Linux, Git, Python data structures
Week 3 - Coordinate frames, transforms, kinematics
Week 4 - ROS 2 fundamentals -- nodes, topics, messages
Week 5 - URDF, TF2, joint state publishing
Week 6 - MoveIt 2 motion planning
Week 7 - Docker, containers, reproducible environments
Week 8 - Gazebo simulation, sensor integration
Week 9 - Gazebo worlds -- Panda, NAO, Quadcopter, Tugbot

You built the foundation every robotics engineer needs.